- **Database (Progress, Users, Quizzes, CTF Flags): SQLite** locally (or `workerd`'s local D1 emulator) using **Drizzle ORM**. When you move to Cloudflare, it migrates 1:1 to **Cloudflare D1** (serverless SQLite at the edge).
- **Course Content & Labs: MDX (Markdown + JSX)** stored in the repo or served from local storage / **Cloudflare R2** (S3-compatible object storage for downloadable PCAPs, log files, and video). Using MDX lets you embed interactive SecOps components (like simulated SIEM log terminals, regex testers, or packet inspectors) directly inside lesson text.

### 2. Handling Interactive SecOps Labs

Depending on what kind of SecOps training you are building, interactive labs can be structured in two ways that survive the move to Cloudflare:

- **Client-Side / Edge-Simulated Labs (Zero-Cost Scaling):**
  - Simulated SIEM query consoles (e.g., parsing JSON/Syslog datasets in-browser using WebAssembly/DuckDB-Wasm or JavaScript).
  - Interactive incident response playbooks, phishing header analysis, and YARA/Sigma rule validation.
- **Containerized Lab Environments (Hybrid):**
  - Keep the LMS frontend/API on Cloudflare, and spin up ephemeral target/attack containers either on **Cloudflare Containers** or bridged back to your self-hosted Unraid box via **Cloudflare Tunnels (`cloudflared`)** with **Cloudflare Access** authentication.

Copy and paste the full text from that first collapsed bubble whenever you're ready, and I will build out the exact implementation you asked for.

---

*User prompt: Here is the original prompt is provided below, however it might be better to build the site and backend in this pass. Then I can provide a research package for each learning topic which can then be broken into modules. I want to build an elearning website. For right now I will host it locally but long term I may want to host it on Cloudflare either as a page or worker. The curriculum for the site will be over multiple domains of cyber security. It should cover penetration testing, threat hunting, and malware (Discovery, blast chamber analysis, reverse engineering, remediation). Each topic should be thoroughly researched and the teaching should be designed for someone with a strong technology background but with no direct experience in these specific fields. Modules for each course should be interesting, interactive, and help progress the user from fundamentals to intermediate level competency.*

Response: Splitting this into **Phase 1 (Platform & Backend Architecture)** and **Phase 2 (Research-Driven Curriculum Ingestion)** is the cleanest way to build this. By decoupling the learning engine from the curriculum data, you can feed me raw research packages later, and I can output structured JSON/Markdown payloads + SQL seed files that drop directly into your platform without touching the core codebase.

Here is the complete, production-ready scaffold designed to run locally right now (via Wrangler's local `workerd` runtime and local SQLite) and deploy to **Cloudflare Workers + D1** later with a single command.

### 1. Architecture & Directory Structure

Cloudflare recently unified Pages and Workers into **Cloudflare Workers with Static Assets**. This means a single project directory serves both your **React + Vite + Tailwind** frontend and your **Hono API + D1 SQLite** backend on the same port locally (`http://localhost:8787`), avoiding CORS headaches.

```
secops-academy/
├── package.json
├── wrangler.toml                # Local & Cloudflare Edge config
├── vite.config.ts
├── index.html
├── migrations/
│   ├── 0001_schema.sql          # Relational LMS + Interactive Lab tables
│   └── 0002_seed_starter.sql    # Sample modules across your 3 domains
└── src/
    ├── worker/
    │   └── index.ts             # Hono API (Courses, Modules, Lab/Flag Verifier)
    └── client/
        ├── main.tsx             # React Entry
        └── App.tsx              # Split-pane LMS UI + Interactive Lab Engine
```

### 2. Configuration & Database Schema

**`package.json`**

```
{
  "name": "secops-academy",
  "private": true,
  "version": "0.1.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "db:init:local": "wrangler d1 execute secops-db --local --file=./migrations/0001_schema.sql && wrangler d1 execute s
    "preview": "npm run build && wrangler dev",
```

```
    "deploy": "npm run build && wrangler deploy"
  },
  "dependencies": {
    "hono": "^4.6.0",
    "lucide-react": "^0.454.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@cloudflare/vite-plugin": "^1.0.0",
    "@types/react": "^18.3.1",
    "@types/react-dom": "^18.3.1",
    "@vitejs/plugin-react": "^4.3.3",
    "autoprefixer": "^10.4.20",
    "postcss": "^8.4.47",
    "tailwindcss": "^3.4.14",
    "typescript": "^5.6.3",
    "vite": "^5.4.10",
    "wrangler": "^3.86.0"
  }
}
```

**wrangler.toml**

When running locally, Wrangler automatically creates a local SQLite database inside `.wrangler/state/v3/d1`.

```
name = "secops-academy"
main = "src/worker/index.ts"
compatibility_date = "2026-03-01"

[assets]
directory = "./dist"
not_found_handling = "single-page-application"
binding = "ASSETS"

[[d1_databases]]
binding = "DB"
database_name = "secops-db"
database_id = "local-dev-db" # Replace with Cloudflare D1 UUID when deploying to edge
```

**migrations/0001_schema.sql**

This schema is specifically designed to accept modular research packages and drive interactive labs (CLI emulators, SIEM log hunters, and static/dynamic malware analyzers).

```
DROP TABLE IF EXISTS user_progress;
DROP TABLE IF EXISTS modules;
DROP TABLE IF EXISTS tracks;

-- 1. Core Domains (Pentesting, Threat Hunting, Malware Analysis)
CREATE TABLE tracks (
    id TEXT PRIMARY KEY,
    slug TEXT UNIQUE NOT NULL,
    title TEXT NOT NULL,
    subtitle TEXT NOT NULL,
    description TEXT NOT NULL,
    accent_color TEXT NOT NULL,
    order_index INTEGER NOT NULL
);

-- 2. Curriculum Modules (Theory + Interactive Lab Spec)
CREATE TABLE modules (
    id TEXT PRIMARY KEY,
    track_id TEXT NOT NULL,
    slug TEXT UNIQUE NOT NULL,
    title TEXT NOT NULL,
    difficulty TEXT CHECK(difficulty IN ('Fundamentals', 'Intermediate', 'Advanced')) NOT NULL,
    order_index INTEGER NOT NULL,
    estimated_minutes INTEGER NOT NULL,
    theory_markdown TEXT NOT NULL,
    -- Lab Types: 'terminal' (CLI/Pentest), 'siem' (Threat Hunting), 'hex_inspector' (Malware RE)
    lab_type TEXT CHECK(lab_type IN ('terminal', 'siem', 'hex_inspector')) NOT NULL,
    lab_briefing TEXT NOT NULL,
    lab_config_json TEXT NOT NULL,      -- JSON payload that hydrates the interactive lab UI
```

```
    expected_flag TEXT NOT NULL,        -- Verified server-side so answers aren't leaked in client bundle
    hint TEXT NOT NULL,
    FOREIGN KEY (track_id) REFERENCES tracks(id)
);

-- 3. Learner Progress & Telemetry
CREATE TABLE user_progress (
    module_id TEXT PRIMARY KEY,
    status TEXT CHECK(status IN ('in_progress', 'completed')) NOT NULL,
    attempts INTEGER DEFAULT 1,
    completed_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (module_id) REFERENCES modules(id)
);
```

### 3. Backend API (`src/worker/index.ts`)

Built on **Hono**, this backend serves curriculum tracks, strips sensitive solution flags before sending lab payloads to the browser, verifies lab submissions server-side, and records learner progression in SQLite/D1.

```
import { Hono } from 'hono';
import { cors } from 'hono/cors';

type Bindings = {
  DB: D1Database;
};

const app = new Hono<{ Bindings: Bindings }>();

app.use('/api/*', cors());

// 1. Get All Tracks & Module Overview with Completion Status
app.get('/api/curriculum', async (c) => {
  const { results: tracks } = await c.env.DB.prepare(
    `SELECT * FROM tracks ORDER BY order_index ASC`
  ).all();

  const { results: modules } = await c.env.DB.prepare(
    `SELECT m.id, m.track_id, m.slug, m.title, m.difficulty, m.order_index,
            m.estimated_minutes, m.lab_type, COALESCE(p.status, 'locked') as status
     FROM modules m
     LEFT JOIN user_progress p ON m.id = p.module_id
     ORDER BY m.order_index ASC`
  ).all();

  return c.json({ tracks, modules });
});

// 2. Get Single Module Content & Safe Lab Payload (Excludes expected_flag)
app.get('/api/modules/:slug', async (c) => {
  const slug = c.req.param('slug');
  const moduleData = await c.env.DB.prepare(
    `SELECT m.id, m.track_id, m.slug, m.title, m.difficulty, m.estimated_minutes,
            m.theory_markdown, m.lab_type, m.lab_briefing, m.lab_config_json, m.hint,
            COALESCE(p.status, 'unstarted') as status
     FROM modules m
     LEFT JOIN user_progress p ON m.id = p.module_id
     WHERE m.slug = ?`
  ).bind(slug).first();

  if (!moduleData) return c.json({ error: 'Module not found' }, 404);

  return c.json({
    ...moduleData,
    lab_config: JSON.parse(moduleData.lab_config_json as string)
  });
});

// 3. Verify Interactive Lab Flag / Answer Submission
app.post('/api/modules/:id/verify', async (c) => {
  const moduleId = c.req.param('id');
  const { submission } = await c.req.json<{ submission: string }>();

  const record = await c.env.DB.prepare(
    `SELECT expected_flag FROM modules WHERE id = ?`
```

```
  ).bind(moduleId).first<{ expected_flag: string }>();

  if (!record) return c.json({ error: 'Module not found' }, 404);

  const isCorrect =
    submission.trim().toLowerCase() === record.expected_flag.trim().toLowerCase();

  if (isCorrect) {
    await c.env.DB.prepare(
      `INSERT INTO user_progress (module_id, status, attempts, completed_at)
       VALUES (?, 'completed', 1, CURRENT_TIMESTAMP)
       ON CONFLICT(module_id) DO UPDATE SET
         status = 'completed',
         attempts = user_progress.attempts + 1,
         completed_at = CURRENT_TIMESTAMP`
    ).bind(moduleId).run();
  }

  return c.json({
    correct: isCorrect,
    message: isCorrect
      ? 'Verification successful. Telemetry logged and module marked complete.'
      : 'Verification failed. Check your telemetry output and try again.'
  });
});

// 4. Ingestion Endpoint for Future Research Packages (Local Admin Use)
app.post('/api/admin/ingest-module', async (c) => {
  const body = await c.req.json();
  await c.env.DB.prepare(
    `INSERT OR REPLACE INTO modules
     (id, track_id, slug, title, difficulty, order_index, estimated_minutes, theory_markdown, lab_type, lab_briefing, la
     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`
  ).bind(
    body.id, body.track_id, body.slug, body.title, body.difficulty,
    body.order_index, body.estimated_minutes, body.theory_markdown,
    body.lab_type, body.lab_briefing, JSON.stringify(body.lab_config),
    body.expected_flag, body.hint
  ).run();

  return c.json({ status: 'ingested', id: body.id });
});

export default app;
```

### 4. Interactive Frontend Engine (`src/client/App.tsx`)

Because your target learner already has a strong IT/systems background, the UI avoids hand-holding and uses a **Split-Screen SecOps Workbench**:

- **Left Pane:** Architectural & technical theory bridging existing IT knowledge (e.g., TCP/IP, Windows internals, Linux processes, HTTP) to offensive/defensive concepts.
- **Right Pane:** A polymorphic interactive lab engine that automatically renders one of three lab environments based on the module's `lab_type`:
    1. `terminal`: Simulated command-line environment for Penetration Testing (reconnaissance, enumeration, exploit verification).
    2. `siem`: Interactive log query & event inspection table for Threat Hunting (Sysmon, Process Creation, network beacons).
    3. `hex_inspector`: Blast-chamber & static/dynamic artifact analyzer for Malware Analysis (PE headers, strings, API imports, sandbox detonation trees, and remediation rules).

```
import React, { useState, useEffect } from 'react';
import { Shield, Crosshair, Bug, Terminal, Search, Cpu, CheckCircle2, HelpCircle, ChevronRight } from 'lucide-react';

export default function App() {
  const [curriculum, setCurriculum] = useState<{ tracks: any[]; modules: any[] }>({ tracks: [], modules: [] });
  const [activeTrackId, setActiveTrackId] = useState<string>('track-pentest');
  const [activeModule, setActiveModule] = useState<any | null>(null);
  const [submission, setSubmission] = useState('');
  const [feedback, setFeedback] = useState<{ correct?: boolean; message?: string } | null>(null);
  const [showHint, setShowHint] = useState(false);

  // Interactive Lab State
  const [terminalHistory, setTerminalHistory] = useState<Array<{ cmd: string; out: string }>>([]);
```

```
const [cmdInput, setCmdInput] = useState('');
const [siemFilter, setSiemFilter] = useState('');

const fetchCurriculum = async () => {
  const res = await fetch('/api/curriculum');
  const data = await res.json();
  setCurriculum(data);
  if (!activeModule && data.modules.length > 0) {
    loadModule(data.modules[0].slug);
  }
};

const loadModule = async (slug: string) => {
  const res = await fetch(`/api/modules/${slug}`);
  const data = await res.json();
  setActiveModule(data);
  setFeedback(null);
  setSubmission('');
  setShowHint(false);
  setTerminalHistory([]);
  setSiemFilter('');
};

useEffect(() => {
  fetchCurriculum();
}, []);

const handleTerminalCommand = (e: React.FormEvent) => {
  e.preventDefault();
  if (!cmdInput.trim() || !activeModule) return;
  const cleanCmd = cmdInput.trim();
  const responses = activeModule.lab_config.commands || {};
  const output = responses[cleanCmd] ||
    (cleanCmd === 'help' ? `Available commands: ${Object.keys(responses).join(', ')}` : `bash: ${cleanCmd}: command or
  setTerminalHistory([...terminalHistory, { cmd: cleanCmd, out: output }]);
  setCmdInput('');
};

const verifyFlag = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!activeModule) return;
  const res = await fetch(`/api/modules/${activeModule.id}/verify`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ submission })
  });
  const result = await res.json();
  setFeedback(result);
  if (result.correct) fetchCurriculum();
};

const currentTrackModules = curriculum.modules.filter((m) => m.track_id === activeTrackId);

return (
  <div className="min-h-screen bg-slate-950 text-slate-100 flex flex-col font-sans">
    {/* Top Navbar */}
    <header className="border-b border-slate-800 bg-slate-900/80 px-6 py-3 flex items-center justify-between">
      <div className="flex items-center gap-3">
        <Shield className="w-6 h-6 text-emerald-400" />
        <span className="font-bold tracking-wider text-lg">SECOPS // ACADEMY</span>
        <span className="text-xs bg-slate-800 text-slate-400 px-2 py-0.5 rounded border border-slate-700">
          IT-to-SecOps Bridge Curriculum
        </span>
      </div>
      <div className="flex gap-2">
        {curriculum.tracks.map((t) => (
          <button
            key={t.id}
            onClick={() => {
              setActiveTrackId(t.id);
              const firstMod = curriculum.modules.find((m) => m.track_id === t.id);
              if (firstMod) loadModule(firstMod.slug);
            }}
            className={`px-3 py-1.5 rounded text-sm font-medium transition ${
```

```
          activeTrackId === t.id
            ? 'bg-emerald-500/15 text-emerald-300 border border-emerald-500/40'
            : 'text-slate-400 hover:text-slate-200 hover:bg-slate-800'
        }`}
      >
        {t.title}
      </button>
    ))}
  </div>
</header>

{/* Main Split Workbench */}
<div className="flex-1 grid grid-cols-12 overflow-hidden">
  {/* Sidebar: Track Modules */}
  <aside className="col-span-2 border-r border-slate-800 bg-slate-900/40 p-4 space-y-2 overflow-y-auto">
    <div className="text-xs font-semibold uppercase tracking-wider text-slate-500 mb-3">
      Track Modules
    </div>
    {currentTrackModules.map((m) => (
      <button
        key={m.id}
        onClick={() => loadModule(m.slug)}
        className={`w-full text-left p-2.5 rounded-lg border text-sm transition flex items-center justify-between
          activeModule?.id === m.id
            ? 'bg-slate-800 border-emerald-500/50 text-white'
            : 'border-transparent hover:bg-slate-900 text-slate-400'
        }`}
      >
        <div>
          <div className="font-medium leading-snug">{m.title}</div>
          <div className="text-[11px] text-slate-500 mt-0.5">{m.difficulty} • {m.estimated_minutes}m</div>
        </div>
        {m.status === 'completed' && <CheckCircle2 className="w-4 h-4 text-emerald-400 shrink-0" />}
      </button>
    ))}
  </aside>

  {/* Center Pane: Technical Theory */}
  <main className="col-span-5 border-r border-slate-800 p-6 overflow-y-auto space-y-4">
    {activeModule && (
      <>
        <div className="flex items-center gap-2 text-xs text-emerald-400 uppercase tracking-wider font-semibold">
          <span>{activeModule.difficulty}</span>
          <ChevronRight className="w-3 h-3" />
          <span>{activeModule.estimated_minutes} Min Read + Lab</span>
        </div>
        <h1 className="text-2xl font-bold text-white">{activeModule.title}</h1>
        <div className="prose prose-invert max-w-none text-slate-300 text-sm leading-relaxed whitespace-pre-line">
          {activeModule.theory_markdown}
        </div>
      </>
    )}
  </main>

  {/* Right Pane: Polymorphic Interactive Lab + Verification */}
  <section className="col-span-5 bg-slate-950 p-6 flex flex-col justify-between overflow-y-auto">
    {activeModule && (
      <div className="space-y-4">
        <div className="bg-slate-900 border border-slate-800 rounded-lg p-4">
          <div className="text-xs uppercase tracking-wider text-amber-400 font-semibold mb-1">
            Interactive Lab Briefing ({activeModule.lab_type.toUpperCase()})
          </div>
          <p className="text-sm text-slate-300">{activeModule.lab_briefing}</p>
        </div>

        {/* LAB ENGINE 1: CLI TERMINAL (Penetration Testing) */}
        {activeModule.lab_type === 'terminal' && (
          <div className="bg-black border border-slate-800 rounded-lg p-4 font-mono text-xs h-80 flex flex-col jus
            <div className="space-y-2">
              <div className="text-slate-500">Connected to lab target container. Type 'help' for scoped tools.</di
              {terminalHistory.map((item, idx) => (
                <div key={idx}>
                  <div className="text-emerald-400">operator@kali-lab:~$ {item.cmd}</div>
                  <pre className="text-slate-300 whitespace-pre-wrap mt-1">{item.out}</pre>
```

```
                </div>
              ))}
            </div>
            <form onSubmit={handleTerminalCommand} className="flex items-center gap-2 pt-2 border-t border-slate-9
              <span className="text-emerald-400">operator@kali-lab:~$</span>
              <input
                value={cmdInput}
                onChange={(e) => setCmdInput(e.target.value)}
                className="bg-transparent flex-1 focus:outline-none text-white"
                placeholder="Enter command..."
              />
            </form>
          </div>
        )}

        {/* LAB ENGINE 2: SIEM LOG HUNTER (Threat Hunting) */}
        {activeModule.lab_type === 'siem' && (
          <div className="bg-slate-900 border border-slate-800 rounded-lg p-4 space-y-3">
            <div className="flex items-center gap-2 bg-slate-950 border border-slate-700 px-3 py-1.5 rounded">
              <Search className="w-4 h-4 text-slate-400" />
              <input
                value={siemFilter}
                onChange={(e) => setSiemFilter(e.target.value)}
                placeholder="Filter Sysmon/EDR telemetry (e.g., powershell, 4688, encoded)..."
                className="bg-transparent text-xs w-full focus:outline-none text-white font-mono"
              />
            </div>
            <div className="max-h-64 overflow-y-auto space-y-1.5 font-mono text-xs">
              {(activeModule.lab_config.logs || [])
                .filter((log: any) => JSON.stringify(log).toLowerCase().includes(siemFilter.toLowerCase()))
                .map((log: any, idx: number) => (
                  <div key={idx} className="p-2 bg-slate-950 rounded border border-slate-800 text-slate-300">
                    <span className="text-sky-400">[{log.timestamp}]</span>{' '}
                    <span className="text-amber-300">EventID={log.event_id}</span>{' '}
                    <span className="text-emerald-300">Image={log.image}</span>
                    <div className="text-slate-400 mt-1 break-all">CmdLine: {log.command_line}</div>
                  </div>
                ))}
            </div>
          </div>
        )}

        {/* LAB ENGINE 3: BLAST CHAMBER & RE INSPECTOR (Malware Analysis) */}
        {activeModule.lab_type === 'hex_inspector' && (
          <div className="bg-slate-900 border border-slate-800 rounded-lg p-4 space-y-3 font-mono text-xs">
            <div className="text-rose-400 font-semibold">SAMPLE: {activeModule.lab_config.sample_name}</div>
            <div className="grid grid-cols-2 gap-2">
              <div className="p-2 bg-slate-950 rounded border border-slate-800">
                <div className="text-slate-500 mb-1">Suspicious API Imports (IAT)</div>
                {(activeModule.lab_config.imports || []).map((imp: string, i: number) => (
                  <div key={i} className="text-amber-300">{imp}</div>
                ))}
              </div>
              <div className="p-2 bg-slate-950 rounded border border-slate-800">
                <div className="text-slate-500 mb-1">Extracted Strings / Decoded IOCs</div>
                {(activeModule.lab_config.strings || []).map((str: string, i: number) => (
                  <div key={i} className="text-emerald-300 break-all">{str}</div>
                ))}
              </div>
            </div>
            <div className="p-2 bg-slate-950 rounded border border-slate-800">
              <div className="text-slate-500 mb-1">Blast Chamber Detonation Telemetry</div>
              <div className="text-slate-300 whitespace-pre-wrap">{activeModule.lab_config.sandbox_trace}</div>
            </div>
          </div>
        )}
      </div>
    )}

    {/* Flag / Objective Verification Footer */}
    {activeModule && (
      <div className="pt-4 border-t border-slate-800 space-y-3">
        <form onSubmit={verifyFlag} className="flex gap-2">
          <input
```

```
                  value={submission}
                  onChange={(e) => setSubmission(e.target.value)}
                  placeholder="Submit finding / flag / IOC..."
                  className="flex-1 bg-slate-900 border border-slate-700 rounded px-3 py-2 text-sm text-white font-mono
                />
                <button
                  type="submit"
                  className="bg-emerald-600 hover:bg-emerald-500 text-white font-medium px-4 py-2 rounded text-sm transi
                >
                  Verify
                </button>
                <button
                  type="button"
                  onClick={() => setShowHint(!showHint)}
                  className="p-2 border border-slate-700 rounded text-slate-400 hover:text-white"
                  title="Toggle Hint"
                >
                  <HelpCircle className="w-5 h-5" />
                </button>
              </form>

              {showHint && (
                <div className="text-xs bg-amber-500/10 border border-amber-500/30 text-amber-200 p-2.5 rounded">
                  <strong>Hint:</strong> {activeModule.hint}
                </div>
              )}

              {feedback && (
                <div
                  className={`text-xs p-2.5 rounded border ${
                    feedback.correct
                      ? 'bg-emerald-500/10 border-emerald-500/30 text-emerald-300'
                      : 'bg-rose-500/10 border-rose-500/30 text-rose-300'
                  }`}
                >
                  {feedback.message}
                </div>
              )}
            </div>
          )}
        </section>
      </div>
    </div>
  );
}
```

### 5. Starter Seed File (`migrations/0002_seed_starter.sql`)

To verify all three interactive lab engines immediately on your local build, this starter seed loads one foundational module for each of your three domains:

1. **Penetration Testing (`terminal` lab):** Service Enumeration & Banner Analysis
2. **Threat Hunting (`siem` lab):** Hunting Living-off-the-Land (LotL) PowerShell Execution in Sysmon
3. **Malware Analysis (`hex_inspector` lab):** Blast Chamber Triage, Process Injection IAT Analysis & Remediation

```
INSERT INTO tracks (id, slug, title, subtitle, description, accent_color, order_index) VALUES
('track-pentest', 'penetration-testing', 'Penetration Testing', 'Attack Surface & Exploitation', 'Transition from system
('track-hunting', 'threat-hunting', 'Threat Hunting', 'Hypothesis-Driven Detection', 'Move beyond reactive alerts: analy
('track-malware', 'malware-analysis', 'Malware Engineering', 'Discovery, Blast Chamber, RE & Remediation', 'End-to-end m

INSERT INTO modules (id, track_id, slug, title, difficulty, order_index, estimated_minutes, theory_markdown, lab_type, l
(
  'mod-pt-101',
  'track-pentest',
  'service-enumeration-fundamentals',
  '101: From Port States to Service Misconfigurations',
  'Fundamentals',
  1,
  15,
  'As an experienced technologist, you already use port checks to verify service availability. In Penetration Testing, w
  'terminal',
  'Enumerate open services on 10.10.14.22, inspect the exposed SMB shares using anonymous authentication, and submit the
  '{"commands": {"nmap -sV 10.10.14.22": "PORT    STATE SERVICE VERSION\n22/tcp  open  ssh     OpenSSH 9.2p1\n445/tcp op
```

```
  'FLAG{SMB_NULL_SESSION_EXPOSED}',
  'Run `help` to see available commands, or start with `nmap -sV 10.10.14.22` followed by `smbclient -L //10.10.14.22 -N
),
(
  'mod-th-101',
  'track-hunting',
  'lotl-powershell-hunting',
  '101: Hunting Encoded Living-off-the-Land (LotL) Execution',
  'Fundamentals',
  1,
  20,
  'Attackers rarely drop noisy custom binaries when native OS administration tools (`powershell.exe`, `rundll32.exe`, `c
  'siem',
  'Filter the Sysmon Event ID 1 process creation logs below to locate the anomalous PowerShell execution. Submit the exa
  '{"logs": [{"timestamp": "2026-10-16T18:01:12Z", "event_id": 1, "image": "C:\\\\Windows\\\\System32\\\\svchost.exe", "
  '2026-10-16T18:04:49Z',
  'Filter by `powershell` or `-Enc` in the SIEM search bar and copy the timestamp inside the brackets.'
),
(
  'mod-mw-101',
  'track-malware',
  'blast-chamber-triage-and-iat',
  '101: Discovery, Blast Chamber Detonation & IAT Triage',
  'Fundamentals',
  1,
  25,
  'Malware analysis follows four disciplined phases:\n1. **Discovery:** Identifying anomalous binaries via entropy, unsi
  'hex_inspector',
  'Inspect the static IAT imports, extracted strings, and Blast Chamber detonation trace for `updater_svc.exe`. Identify
  '{"sample_name": "updater_svc.exe (SHA256: 9f86d081884c7d659a2feaa0c55ad015)", "imports": ["KERNEL32.dll!VirtualAllocE
  'telemetry-cdn-check.net',
  'Check the Blast Chamber Detonation Telemetry and Extracted Strings for the external domain name.'
);
```

### 6. How We Will Ingest Your Research Packages (Phase 2 Workflow)

Once you spin this up locally (`npm install` → `npm run db:init:local` → `npm run preview`), you can feed me your research packages topic by topic.

For each research package you provide, I will transform it into:

1. **A Progressive Module Chain** (e.g., Module 1: Fundamentals, Module 2: Intermediate Mechanics, Module 3: Applied Scenario/Remediation) written specifically for a seasoned IT professional transitioning into SecOps.
2. **Interactive Lab Configurations** (`terminal`, `siem`, or `hex_inspector` JSON payloads) tailored to the exact artifacts in your research.
3. **A Drop-in SQL / JSON Ingestion File** that loads the new modules straight into your local SQLite database (and later syncs to Cloudflare D1).

Would you like me to also render a live interactive preview of this frontend in Canvas right now so you can test-drive the three lab engines (`terminal`, `siem`, and `hex_inspector`), or are you ready to drop in the first research package?

---

*User prompt: Yes render a preview*

Response: Below is a self-contained **single-file interactive preview (`secops-preview.html`)** that packs the entire React + Tailwind frontend and an in-memory D1 database simulation into one file.

If you are viewing this in the web interface, click **Preview** on the code block below to test-drive it immediately—or save it locally as `secops-preview.html` and open it in any browser.

### Included in this Interactive Preview:

- **6 Complete Modules (Fundamentals → Intermediate)** across all three tracks (**Penetration Testing**, **Threat Hunting**, and **Malware Engineering**).
- **Clickable Quick-Run Chips & Interactive Sub-Tabs:** Test commands in the Kali terminal with a single click or type them manually, decode Base64 payloads inside the SIEM log hunter, and switch between **Static IAT/Strings**, **Blast Chamber Telemetry**, and **Decompiled Pseudocode / YARA Remediation** in the Malware Inspector.
- **Live JSON/SQL Ingest Modal:** Click **"Ingest Research Package"** in the top right to see how newly generated module JSON payloads dynamically hot-load into the platform.

```
<!DOCTYPE html>
<html lang="en" class="dark">
```

```
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>SECOPS // ACADEMY — Interactive Preview</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://unpkg.com/react@18/umd/react.production.min.js" crossorigin></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.production.min.js" crossorigin></script>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
  <style>
    ::-webkit-scrollbar { width: 6px; height: 6px; }
    ::-webkit-scrollbar-track { background: #090d16; }
    ::-webkit-scrollbar-thumb { background: #1e293b; border-radius: 3px; }
    ::-webkit-scrollbar-thumb:hover { background: #334155; }
  </style>
</head>
<body class="bg-slate-950 text-slate-100 antialiased selection:bg-emerald-500/30 selection:text-emerald-200">
  <div id="root"></div>

  <script type="text/babel">
    const { useState, useEffect, useRef } = React;

    // --- INITIAL SEED DATA (SIMULATING CLOUDFLARE D1 / LOCAL SQLITE) ---
    const INITIAL_TRACKS = [
      {
        id: 'track-pentest',
        slug: 'penetration-testing',
        title: 'Penetration Testing',
        subtitle: 'Attack Surface & Exploitation',
        badge: 'OFFENSIVE',
        color: 'emerald'
      },
      {
        id: 'track-hunting',
        slug: 'threat-hunting',
        title: 'Threat Hunting',
        subtitle: 'Hypothesis-Driven Telemetry Analysis',
        badge: 'DEFENSIVE',
        color: 'sky'
      },
      {
        id: 'track-malware',
        slug: 'malware-analysis',
        title: 'Malware Engineering',
        subtitle: 'Discovery, Blast Chamber, RE & Remediation',
        badge: 'MALWARE LAB',
        color: 'rose'
      }
    ];

    const INITIAL_MODULES = [
      // ==================== TRACK 1: PENETRATION TESTING ====================
      {
        id: 'mod-pt-101',
        track_id: 'track-pentest',
        slug: 'service-enumeration-fundamentals',
        title: '101: From Port States to Service Misconfigurations',
        difficulty: 'Fundamentals',
        estimated_minutes: 15,
        lab_type: 'terminal',
        theory_sections: [
          {
            heading: 'The Systems-to-Offensive Mindset Shift',
            body: 'As an experienced systems administrator or infrastructure engineer, you use port checks (`nc`, `Test-
          },
          {
            heading: 'SMB Null Sessions & CI/CD Staging Leakage',
            body: 'Server Message Block (SMB on TCP/445) frequently bridges Windows and Linux environments via Samba. Wh
          },
          {
            heading: 'Lab Methodology',
            body: '1. Fingerprinting service versions (`nmap -sV`).\n2. Listing SMB shares anonymously (`smbclient -L`).
          }
        ],
        lab_briefing: 'Enumerate open services on target 10.10.14.22, inspect the exposed SMB shares using anonymous aut
```

```
  lab_config: {
    target_ip: '10.10.14.22',
    quick_commands: [
      'nmap -sV 10.10.14.22',
      'smbclient -L //10.10.14.22 -N',
      'smbclient //10.10.14.22/dev-staging -N -c "ls"',
      'smbclient //10.10.14.22/dev-staging -N -c "get deploy.env -"'
    ],
    commands: {
      'nmap -sV 10.10.14.22': `Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-16 18:10 EDT\nNmap scan report f
      'smbclient -L //10.10.14.22 -N': `Anonymous login successful\n\n\tSharename       Type      Comment\n\t-----
      'smbclient //10.10.14.22/dev-staging -N -c "ls"': `Anonymous login successful\n  .
      'smbclient //10.10.14.22/dev-staging -N -c "get deploy.env -"': `Anonymous login successful\n# Staging Runne
    }
  },
  expected_flag: 'FLAG{SMB_NULL_SESSION_EXPOSED}',
  hint: 'Click or run the four quick commands in sequence, ending with `smbclient //10.10.14.22/dev-staging -N -c
},
{
  id: 'mod-pt-201',
  track_id: 'track-pentest',
  slug: 'ssrf-cloud-metadata-pivoting',
  title: '201: SSRF to Cloud IMDS Credential Extraction',
  difficulty: 'Intermediate',
  estimated_minutes: 25,
  lab_type: 'terminal',
  theory_sections: [
    {
      heading: 'Why SSRF Bypasses Perimeter Firewalls',
      body: 'In modern hybrid cloud architectures, perimeter WAFs and security groups block external access to int
    },
    {
      heading: 'Targeting the Instance Metadata Service (169.254.169.254)',
      body: 'Cloud providers expose an Instance Metadata Service (IMDS) on the link-local address `169.254.169.254
    }
  ],
  lab_briefing: 'The internal webhook proxy at http://10.10.14.55:8080/proxy?url= fetches remote status pages. Abu
  lab_config: {
    target_ip: '10.10.14.55',
    quick_commands: [
      'curl -s "http://10.10.14.55:8080/proxy?url=http://localhost:8080/health"',
      'curl -s "http://10.10.14.55:8080/proxy?url=http://169.254.169.254/latest/meta-data/iam/security-credentials
      'curl -s "http://10.10.14.55:8080/proxy?url=http://169.254.169.254/latest/meta-data/iam/security-credentials
    ],
    commands: {
      'curl -s "http://10.10.14.55:8080/proxy?url=http://localhost:8080/health"': `{"status":"ok","service":"inter
      'curl -s "http://10.10.14.55:8080/proxy?url=http://169.254.169.254/latest/meta-data/iam/security-credentials
      'curl -s "http://10.10.14.55:8080/proxy?url=http://169.254.169.254/latest/meta-data/iam/security-credentials
    }
  },
  expected_flag: 'FLAG{IMDS_SSRF_ROLE_COMPROMISED}',
  hint: 'Query the `secops-ec2-role` metadata endpoint via the `/proxy?url=` parameter and submit the value in the
},

// ==================== TRACK 2: THREAT HUNTING ====================
{
  id: 'mod-th-101',
  track_id: 'track-hunting',
  slug: 'lotl-powershell-hunting',
  title: '101: Hunting Encoded Living-off-the-Land (LotL) Execution',
  difficulty: 'Fundamentals',
  estimated_minutes: 20,
  lab_type: 'siem',
  theory_sections: [
    {
      heading: 'Why Signature Alerts Miss Living-off-the-Land (LotL)',
      body: 'Adversaries rarely drop noisy custom binaries during initial execution when native Windows binaries (
    },
    {
      heading: 'Pivoting on Sysmon Event ID 1 (Process Creation)',
      body: 'Threat hunters focus on behavioral anomalies rather than file hashes:\n• **Parent-Child Anomalies:**
    }
  ],
  lab_briefing: 'Filter the Sysmon Event ID 1 telemetry stream to locate the obfuscated PowerShell execution spawn
```

```
    lab_config: {
      logs: [
        {
          timestamp: '2026-10-16T18:01:12Z',
          host: 'WKSTN-FIN-04',
          event_id: 1,
          parent_image: 'C:\\Windows\\System32\\services.exe',
          image: 'C:\\Windows\\System32\\svchost.exe',
          command_line: 'svchost.exe -k netsvcs -p -s Schedule'
        },
        {
          timestamp: '2026-10-16T18:03:05Z',
          host: 'WKSTN-FIN-04',
          event_id: 1,
          parent_image: 'C:\\Windows\\explorer.exe',
          image: 'C:\\Program Files\\Microsoft Office\\root\\Office16\\WINWORD.EXE',
          command_line: '"C:\\Program Files\\Microsoft Office\\root\\Office16\\WINWORD.EXE" /n "C:\\Users\\jdoe\\Dow
        },
        {
          timestamp: '2026-10-16T18:04:49Z',
          host: 'WKSTN-FIN-04',
          event_id: 1,
          parent_image: 'C:\\Program Files\\Microsoft Office\\root\\Office16\\WINWORD.EXE',
          image: 'C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe',
          command_line: 'powershell.exe -NoP -NonI -W Hidden -Exec Bypass -Enc aAB0AHQAcAA6AC8ALwAxADAALgAxADAALgAxA
        },
        {
          timestamp: '2026-10-16T18:06:02Z',
          host: 'WKSTN-FIN-04',
          event_id: 1,
          parent_image: 'C:\\Program Files\\Google\\Chrome\\Application\\chrome.exe',
          image: 'C:\\Program Files\\Google\\Chrome\\Application\\chrome.exe',
          command_line: 'chrome.exe --type=renderer --lang=en-US'
        }
      ]
    },
    expected_flag: 'http://10.10.19.99/stage2.ps1',
    hint: 'Filter by `powershell` or `WINWORD`, click "Decode Base64" on the `-Enc` string, and submit the decoded U
  },
  {
    id: 'mod-th-201',
    track_id: 'track-hunting',
    slug: 'wmi-persistence-consumer-bindings',
    title: '201: Hunting Fileless WMI Event Subscription Persistence',
    difficulty: 'Intermediate',
    estimated_minutes: 25,
    lab_type: 'siem',
    theory_sections: [
      {
        heading: 'Beyond Registry Run Keys',
        body: 'Intermediate adversaries avoid `HKLM\\...\\CurrentVersion\\Run` because EDR agents heavily monitor re
      },
      {
        heading: 'The WMI Persistence Triad (Sysmon Events 19, 20, 21)',
        body: '1. `__EventFilter` (Event ID 19): Defines the trigger (e.g., 5 minutes after system boot).\n2. `__Eve
      }
    ],
    lab_briefing: 'Inspect the Sysmon WMI telemetry (Events 19, 20, 21) and process logs across SRV-AD-02. Identify
    lab_config: {
      logs: [
        {
          timestamp: '2026-10-16T19:12:01Z',
          host: 'SRV-AD-02',
          event_id: 19,
          parent_image: 'WmiEventFilter',
          image: 'ROOT\\subscription:__EventFilter',
          command_line: 'FilterName="SCM_Health_Trigger" Query="SELECT * FROM __InstanceModificationEvent WITHIN 60
        },
        {
          timestamp: '2026-10-16T19:12:04Z',
          host: 'SRV-AD-02',
          event_id: 20,
          parent_image: 'WmiEventConsumer',
          image: 'ROOT\\subscription:CommandLineEventConsumer',
```

```
        command_line: 'ConsumerName="SysHealthUpdaterC2" Destination="rundll32.exe C:\\ProgramData\\uso_cache.dat,
      },
      {
        timestamp: '2026-10-16T19:12:05Z',
        host: 'SRV-AD-02',
        event_id: 21,
        parent_image: 'WmiFilterToConsumerBinding',
        image: 'ROOT\\subscription:__FilterToConsumerBinding',
        command_line: 'Consumer="CommandLineEventConsumer.Name=\\"SysHealthUpdaterC2\\"" Filter="__EventFilter.Nam
      }
    ]
  },
  expected_flag: 'SysHealthUpdaterC2',
  hint: 'Look at Sysmon Event ID 20 (`CommandLineEventConsumer`) and submit the `ConsumerName` value (`SysHealthUp
},

// ==================== TRACK 3: MALWARE ENGINEERING ====================
{
  id: 'mod-mw-101',
  track_id: 'track-malware',
  slug: 'blast-chamber-triage-and-iat',
  title: '101: Discovery, Blast Chamber Detonation & IAT Triage',
  difficulty: 'Fundamentals',
  estimated_minutes: 25,
  lab_type: 'hex_inspector',
  theory_sections: [
    {
      heading: 'The 4-Stage Malware Engineering Lifecycle',
      body: '1. **Discovery:** Flagging anomalous binaries via high section entropy (packed code), unsigned PE hea
    },
    {
      heading: 'Recognizing Process Injection in the IAT',
      body: 'Before even opening a disassembler, inspecting a PE file\'s Import Address Table reveals its capabili
    }
  ],
  lab_briefing: 'Examine the static PE metadata, Blast Chamber detonation trace, and decompiled routine for `updat
  lab_config: {
    sample_name: 'updater_svc.exe (SHA256: 9f86d081884c7d659a2feaa0c55ad015)',
    entropy: '7.42 (High - Packed .rsrc section)',
    imports: [
      'KERNEL32.dll!OpenProcess',
      'KERNEL32.dll!VirtualAllocEx',
      'KERNEL32.dll!WriteProcessMemory',
      'KERNEL32.dll!CreateRemoteThread',
      'WININET.dll!InternetOpenUrlA'
    ],
    strings: [
      'Software\\Microsoft\\Windows\\CurrentVersion\\Run',
      'Global\\Mtx_Sync_881',
      'Mozilla/5.0 (Windows NT 10.0; Win64; x64)',
      'https://telemetry-cdn-check.net/gate.php'
    ],
    sandbox_trace: `[00:01.12]EXEC: Spawned updater_svc.exe (PID 4120)\n[00:01.45] SYNC: Created Mutex "Global\\Mt
    decompiled_c: `int __cdecl main() {\n  HANDLE hMutex = CreateMutexA(NULL, TRUE, "Global\\\\Mtx_Sync_881");\n
    remediation_yara: `rule Malware_Win32_UpdaterSvc_Injector {\n  meta:\n    author = "SecOps Academy"\n    sever
  },
  expected_flag: 'telemetry-cdn-check.net',
  hint: 'Inspect the Extracted Strings or Blast Chamber Detonation tab and submit the domain name (`telemetry-cdn-
},
{
  id: 'mod-mw-201',
  track_id: 'track-malware',
  slug: 'xor-config-reverse-engineering-remediation',
  title: '201: Reverse Engineering XOR-Obfuscated Configs & Remediation',
  difficulty: 'Intermediate',
  estimated_minutes: 30,
  lab_type: 'hex_inspector',
  theory_sections: [
    {
      heading: 'Defeating Static String Extraction',
      body: 'To prevent analysts from simply running `strings` on a binary to extract C2 domains and registry keys
    },
    {
      heading: 'Decompilation & Automated Remediation',
```

```
        body: 'By examining the decompiled loop in Ghidra or IDA, you can identify the XOR key constant, decode the
      }
    ],
    lab_briefing: 'Switch to the "Decompiled RE & YARA" tab in the Malware Inspector. Analyze the decompiled XOR dec
    lab_config: {
      sample_name: 'svc_host_helper.dll (SHA256: 4b227777d4dd1fc61c6f884f48641d02)',
      entropy: '6.89 (.data config encrypted)',
      imports: [
        'ADVAPI32.dll!CreateServiceA',
        'ADVAPI32.dll!StartServiceCtrlDispatcherA',
        'WS2_32.dll!connect',
        'KERNEL32.dll!VirtualProtect'
      ],
      strings: [
        '[ENCRYPTED BLOB @ 0x10004020]: 32 3E 3A 29 05 39 35 28 3F 14 3E 36 36',
        'ServiceDll',
        'SYSTEM\\CurrentControlSet\\Services\\WinDefendCoreSvc'
      ],
      sandbox_trace: `[00:00.40] EXEC: rundll32.exe svc_host_helper.dll,InstallSvc\n[00:00.85] MEM : Decrypting 64-b
      decompiled_c: `void __fastcall DecryptConfig(uint8_t *buf, size_t len) {\n  // Single-byte XOR deobfuscation r
      remediation_yara: `rule Malware_Win32_SvcHostHelper_XOR {\n  meta:\n    description = "Detects XOR 0x5A decryp
    },
    expected_flag: '0x5A',
    hint: 'Open the "Decompiled RE & YARA" tab inside the lab pane and check the `uint8_t xor_key` constant (`0x5A`)
  }
];

function App() {
  const [tracks] = useState(INITIAL_TRACKS);
  const [modules, setModules] = useState(INITIAL_MODULES);
  const [completedIds, setCompletedIds] = useState(['mod-pt-101']);
  const [activeTrackId, setActiveTrackId] = useState('track-pentest');
  const [activeModuleId, setActiveModuleId] = useState('mod-pt-101');

  // Interactive Lab States
  const [submission, setSubmission] = useState('');
  const [feedback, setFeedback] = useState(null);
  const [showHint, setShowHint] = useState(false);

  // Terminal State
  const [terminalHistory, setTerminalHistory] = useState([]);
  const [cmdInput, setCmdInput] = useState('');
  const terminalEndRef = useRef(null);

  // SIEM State
  const [siemFilter, setSiemFilter] = useState('');
  const [decodedOutput, setDecodedOutput] = useState('');

  // Malware Inspector Tab State
  const [mwTab, setMwTab] = useState('static'); // 'static' | 'sandbox' | 'decompiled'

  // Research Package Ingestion Modal
  const [showIngestModal, setShowIngestModal] = useState(false);
  const [ingestJson, setIngestJson] = useState('');

  const activeModule = modules.find((m) => m.id === activeModuleId) || modules[0];
  const trackModules = modules.filter((m) => m.track_id === activeTrackId);

  useEffect(() => {
    setFeedback(null);
    setSubmission('');
    setShowHint(false);
    setTerminalHistory([]);
    setSiemFilter('');
    setDecodedOutput('');
    setMwTab('static');
  }, [activeModuleId]);

  useEffect(() => {
    if (terminalEndRef.current) {
      terminalEndRef.current.scrollIntoView({ behavior: 'smooth' });
    }
  }, [terminalHistory]);
```

```
const runTerminalCommand = (rawCmd) => {
  const cleanCmd = rawCmd.trim();
  if (!cleanCmd) return;
  if (cleanCmd === 'clear') {
    setTerminalHistory([]);
    setCmdInput('');
    return;
  }
  const responses = activeModule.lab_config.commands || {};
  let output = responses[cleanCmd];
  if (!output) {
    if (cleanCmd === 'help') {
      output = `Scoped commands available in this lab container:\n  • ${Object.keys(responses).join('\n  • ')}\n
    } else {
      output = `bash: ${cleanCmd}: command outside lab scope or syntax mismatch. Click a Quick-Run command above o
    }
  }
  setTerminalHistory((prev) => [...prev, { cmd: cleanCmd, out: output }]);
  setCmdInput('');
};

const decodeUtf16Base64 = (b64) => {
  try {
    const raw = atob(b64);
    let out = '';
    for (let i = 0; i < raw.length; i++) {
      if (raw.charCodeAt(i) !== 0) out += raw[i];
    }
    setDecodedOutput(out);
  } catch (e) {
    setDecodedOutput('Invalid Base64 string');
  }
};

const handleVerify = (e) => {
  e.preventDefault();
  if (!submission.trim()) return;
  const isCorrect =
    submission.trim().toLowerCase() === activeModule.expected_flag.trim().toLowerCase();

  if (isCorrect) {
    if (!completedIds.includes(activeModule.id)) {
      setCompletedIds([...completedIds, activeModule.id]);
    }
    setFeedback({
      correct: true,
      message: `Verified (${activeModule.expected_flag}). Telemetry recorded and module marked complete.`
    });
  } else {
    setFeedback({
      correct: false,
      message: 'Verification mismatch. Check the artifact output or toggle the Hint icon.'
    });
  }
};

const handleIngestSubmit = (e) => {
  e.preventDefault();
  try {
    const parsed = JSON.parse(ingestJson);
    setModules((prev) => [...prev, parsed]);
    setActiveTrackId(parsed.track_id);
    setActiveModuleId(parsed.id);
    setShowIngestModal(false);
    setIngestJson('');
  } catch (err) {
    alert('Invalid JSON format. Check syntax.');
  }
};

const loadSampleResearchPackage = () => {
  const sample = {
    id: 'mod-mw-301',
    track_id: 'track-malware',
```

```
    slug: 'api-hashing-dynamic-resolution',
    title: '301: Defeating Dynamic API Hashing (ROR13)',
    difficulty: 'Intermediate',
    estimated_minutes: 30,
    lab_type: 'hex_inspector',
    theory_sections: [
      {
        heading: 'Why Advanced Malware Hides the IAT',
        body: 'When malware authors want to hide `VirtualAllocEx` from static IAT inspection, they walk the Proces
      }
    ],
    lab_briefing: 'Inspect the resolved ROR13 hash table in the Decompiled RE tab and submit the API name resolved
    lab_config: {
      sample_name: 'stage3_reflective.bin',
      entropy: '7.81 (Reflective Loader)',
      imports: ['KERNEL32.dll!LoadLibraryA (Only 1 import visible in IAT!)'],
      strings: ['No cleartext Win32 API strings found in binary'],
      sandbox_trace: '[00:00.15] PEB Walk detected -> kernel32.dll export table\n[00:00.22] Resolved ROR13 Hash 0x
      decompiled_c: '// Dynamic API Resolution via PEB + ROR13\nLPVOID api = ResolveApiByHash(0x0726774C); // Reso
      remediation_yara: 'rule Reflective_ROR13_Loader { condition: uint16(0) == 0x5A4D }'
    },
    expected_flag: 'InternetOpenA',
    hint: 'Look at the resolved function name for hash 0x0726774C in the Blast Chamber trace or Decompiled C tab.'
  };
  setIngestJson(JSON.stringify(sample, null, 2));
};

const completionPct = Math.round((completedIds.length / modules.length) * 100);

return (
  <div className="min-h-screen flex flex-col bg-slate-950 text-slate-100">
    {/* Top Navigation Bar */}
    <header className="border-b border-slate-800 bg-slate-900/90 backdrop-blur px-6 py-3 flex flex-wrap items-cent
      <div className="flex items-center gap-3">
        <div className="w-8 h-8 rounded-lg bg-emerald-500/10 border border-emerald-500/40 flex items-center justif
          S//A
        </div>
        <div>
          <div className="flex items-center gap-2">
            <span className="font-bold tracking-wider text-sm md:text-base text-white">SECOPS // ACADEMY</span>
            <span className="text-[11px] font-mono bg-slate-800 text-emerald-300 px-2 py-0.5 rounded border border
              Local D1 + Worker Preview
            </span>
          </div>
          <p className="text-xs text-slate-400 hidden sm:block">
            Multi-Domain Cyber Operations Curriculum for Experienced Technologists
          </p>
        </div>
      </div>

      {/* Domain Track Switcher */}
      <div className="flex items-center gap-2 bg-slate-950 p-1 rounded-lg border border-slate-800">
        {tracks.map((t) => {
          const isActive = activeTrackId === t.id;
          return (
            <button
              key={t.id}
              onClick={() => {
                setActiveTrackId(t.id);
                const firstMod = modules.find((m) => m.track_id === t.id);
                if (firstMod) setActiveModuleId(firstMod.id);
              }}
              className={`px-3 py-1.5 rounded-md text-xs font-medium transition flex items-center gap-2 ${
                isActive
                  ? 'bg-emerald-500/20 text-emerald-300 border border-emerald-500/40 shadow-sm'
                  : 'text-slate-400 hover:text-slate-200 hover:bg-slate-900'
              }`}
            >
              <span>{t.title}</span>
            </button>
          );
        })}
      </div>
```

```
        {/* Progress & Research Ingestion Action */}
        <div className="flex items-center gap-4">
          <div className="hidden lg:flex items-center gap-2 text-xs">
            <span className="text-slate-400">Progress:</span>
            <div className="w-24 h-2 bg-slate-800 rounded-full overflow-hidden">
              <div
                className="h-full bg-emerald-500 transition-all duration-300"
                style={{ width: `${completionPct}%` }}
              />
            </div>
            <span className="font-mono text-emerald-400">{completedIds.length}/{modules.length}</span>
          </div>
          <button
            onClick={() => setShowIngestModal(true)}
            className="text-xs bg-slate-800 hover:bg-slate-700 text-slate-200 border border-slate-700 px-3 py-1.5 ro
          >
            + Ingest Research Package
          </button>
        </div>
      </header>

      {/* Main 3-Pane Split Workbench */}
      <div className="flex-1 grid grid-cols-1 lg:grid-cols-12 overflow-hidden">
        {/* LEFT SIDEBAR: Track Curriculum Tree */}
        <aside className="lg:col-span-3 xl:col-span-2 border-b lg:border-b-0 lg:border-r border-slate-800 bg-slate-9
          <div className="space-y-3">
            <div>
              <div className="text-[11px] font-mono uppercase tracking-wider text-emerald-400">
                Active Domain Track
              </div>
              <h2 className="font-bold text-sm text-white mt-0.5">
                {tracks.find((t) => t.id === activeTrackId)?.title}
              </h2>
              <p className="text-xs text-slate-400 mt-0.5">
                {tracks.find((t) => t.id === activeTrackId)?.subtitle}
              </p>
            </div>

            <div className="pt-2 border-t border-slate-800/80 space-y-2">
              {trackModules.map((m) => {
                const isCompleted = completedIds.includes(m.id);
                const isSelected = activeModule.id === m.id;
                return (
                  <button
                    key={m.id}
                    onClick={() => setActiveModuleId(m.id)}
                    className={`w-full text-left p-3 rounded-lg border transition flex items-start justify-between g
                      isSelected
                        ? 'bg-slate-800/90 border-emerald-500/50 text-white shadow'
                        : 'bg-slate-900/40 border-slate-800/80 hover:bg-slate-900 text-slate-300'
                    }`}
                  >
                    <div className="space-y-1">
                      <div className="text-xs font-semibold leading-snug">{m.title}</div>
                      <div className="flex items-center gap-2 text-[11px]">
                        <span
                          className={`px-1.5 py-0.2 rounded font-mono ${
                            m.difficulty === 'Fundamentals'
                              ? 'bg-sky-500/15 text-sky-300'
                              : 'bg-amber-500/15 text-amber-300'
                          }`}
                        >
                          {m.difficulty}
                        </span>
                        <span className="text-slate-500">{m.estimated_minutes}m</span>
                      </div>
                    </div>
                    <span
                      className={`w-4 h-4 rounded-full flex items-center justify-center text-[10px] shrink-0 mt-0.5
                        isCompleted
                          ? 'bg-emerald-500/20 text-emerald-400 border border-emerald-500/40'
                          : 'border border-slate-700 text-transparent'
                      }`}
                    >
```

```
                ✓
              </span>
            </button>
          );
        })}
      </div>
    </div>

    <div className="hidden lg:block pt-4 border-t border-slate-800/80 text-[11px] text-slate-500 font-mono spa
      <div>RUNTIME: Cloudflare Workerd</div>
      <div>STORAGE: D1 SQLite Schema v1</div>
    </div>
  </aside>

  {/* CENTER PANE: Structured Technical Theory */}
  <main className="lg:col-span-4 xl:col-span-5 border-b lg:border-b-0 lg:border-r border-slate-800 p-6 overflo
    <div className="space-y-2">
      <div className="flex items-center gap-2 text-xs font-mono">
        <span className="px-2 py-0.5 rounded bg-emerald-500/15 text-emerald-300 border border-emerald-500/30">
          {activeModule.difficulty}
        </span>
        <span className="text-slate-500">•</span>
        <span className="text-slate-400">{activeModule.estimated_minutes} Min Module</span>
        <span className="text-slate-500">•</span>
        <span className="text-amber-300 uppercase">Lab: {activeModule.lab_type}</span>
      </div>
      <h1 className="text-xl md:text-2xl font-bold text-white tracking-tight">
        {activeModule.title}
      </h1>
    </div>

    <div className="space-y-5">
      {(activeModule.theory_sections || []).map((sec, i) => (
        <div key={i} className="bg-slate-900/50 border border-slate-800/90 rounded-lg p-4 space-y-2">
          <h3 className="text-sm font-semibold text-emerald-300 tracking-wide">
            {sec.heading}
          </h3>
          <div className="text-sm text-slate-300 leading-relaxed whitespace-pre-line">
            {sec.body}
          </div>
        </div>
      ))}
    </div>
  </main>

  {/* RIGHT PANE: Polymorphic Interactive SecOps Lab */}
  <section className="lg:col-span-5 bg-slate-950 p-6 flex flex-col justify-between overflow-y-auto space-y-4">
    <div className="space-y-4">
      {/* Lab Briefing Banner */}
      <div className="bg-slate-900 border border-slate-800 rounded-lg p-4">
        <div className="flex items-center justify-between mb-1">
          <span className="text-xs font-mono uppercase tracking-wider text-amber-400 font-semibold">
            Interactive Lab Environment // {activeModule.lab_type.toUpperCase()}
          </span>
          <span className="text-[11px] font-mono text-emerald-400 bg-emerald-950/60 px-2 py-0.5 rounded border
            SANDBOX ACTIVE
          </span>
        </div>
        <p className="text-xs md:text-sm text-slate-300 leading-relaxed">
          {activeModule.lab_briefing}
        </p>
      </div>

      {/* ================= LAB ENGINE 1: CLI TERMINAL (Pentesting) ================= */}
      {activeModule.lab_type === 'terminal' && (
        <div className="space-y-2">
          <div className="flex flex-wrap items-center gap-1.5">
            <span className="text-[11px] text-slate-400 font-mono mr-1">Quick-Run:</span>
            {(activeModule.lab_config.quick_commands || []).map((qcmd, i) => (
              <button
                key={i}
                onClick={() => runTerminalCommand(qcmd)}
                className="text-[11px] font-mono bg-slate-900 hover:bg-slate-800 text-emerald-300 border borde
              >
```

```
                $ {qcmd}
              </button>
            ))}
          </div>

          <div className="bg-black border border-slate-800 rounded-lg p-4 font-mono text-xs h-80 flex flex-col
            <div className="space-y-3 overflow-y-auto pr-1">
              <div className="text-slate-500">
                Kali SecOps Container connected. Target scope: {activeModule.lab_config.target_ip}. Click a Qu
              </div>
              {terminalHistory.map((item, idx) => (
                <div key={idx} className="space-y-1">
                  <div className="text-emerald-400">operator@kali-lab:~$ {item.cmd}</div>
                  <pre className="text-slate-300 whitespace-pre-wrap bg-slate-950/80 p-2 rounded border border
                    {item.out}
                  </pre>
                </div>
              ))}
              <div ref={terminalEndRef} />
            </div>

            <form
              onSubmit={(e) => {
                e.preventDefault();
                runTerminalCommand(cmdInput);
              }}
              className="flex items-center gap-2 pt-2 border-t border-slate-900 mt-2"
            >
              <span className="text-emerald-400 shrink-0">operator@kali-lab:~$</span>
              <input
                value={cmdInput}
                onChange={(e) => setCmdInput(e.target.value)}
                className="bg-transparent flex-1 focus:outline-none text-white"
                placeholder="Type command or click a Quick-Run chip above..."
              />
            </form>
          </div>
        </div>
      )}

      {/* ================= LAB ENGINE 2: SIEM LOG HUNTER (Threat Hunting) ================= */}
      {activeModule.lab_type === 'siem' && (
        <div className="bg-slate-900 border border-slate-800 rounded-lg p-4 space-y-3">
          <div className="flex items-center gap-2 bg-slate-950 border border-slate-700 px-3 py-2 rounded">
            <span className="text-xs font-mono text-sky-400">QUERY:</span>
            <input
              value={siemFilter}
              onChange={(e) => setSiemFilter(e.target.value)}
              placeholder="Filter telemetry (e.g., powershell, WINWORD, EventID, ConsumerName)..."
              className="bg-transparent text-xs w-full focus:outline-none text-white font-mono"
            />
            {siemFilter && (
              <button
                onClick={() => setSiemFilter('')}
                className="text-xs text-slate-500 hover:text-slate-300"
              >
                Clear
              </button>
            )}
          </div>

          <div className="max-h-64 overflow-y-auto space-y-2 font-mono text-xs pr-1">
            {(activeModule.lab_config.logs || [])
              .filter((log) =>
                JSON.stringify(log).toLowerCase().includes(siemFilter.toLowerCase())
              )
              .map((log, idx) => {
                const hasBase64 = log.command_line.includes('-Enc ');
                const b64Match = hasBase64 ? log.command_line.split('-Enc ')[1] : null;
                return (
                  <div
                    key={idx}
                    className="p-3 bg-slate-950 rounded border border-slate-800 hover:border-slate-700 space-y
                  >
```

```
                <div className="flex flex-wrap items-center justify-between gap-2">
                  <span className="text-sky-400">{log.timestamp}</span>
                  <span className="bg-amber-500/10 text-amber-300 border border-amber-500/30 px-1.5 py-0.5
                    Sysmon EventID={log.event_id}
                  </span>
                  <span className="text-slate-400">Host={log.host}</span>
                </div>
                <div className="text-slate-400">
                  Parent: <span className="text-slate-200">{log.parent_image}</span>
                </div>
                <div className="text-slate-400">
                  Image: <span className="text-emerald-300">{log.image}</span>
                </div>
                <div className="text-slate-300 bg-slate-900 p-2 rounded border border-slate-800 break-all"
                  CmdLine: {log.command_line}
                </div>
                {b64Match && (
                  <div className="pt-1">
                    <button
                      onClick={() => decodeUtf16Base64(b64Match)}
                      className="text-[11px] bg-sky-500/20 hover:bg-sky-500/30 text-sky-300 border border-
                    >
                       Decode UTF-16LE Base64 Payload
                    </button>
                  </div>
                )}
              </div>
            );
          })}
      </div>

      {decodedOutput && (
        <div className="p-3 bg-emerald-950/30 border border-emerald-500/40 rounded font-mono text-xs space
          <div className="text-emerald-400 font-semibold">
            Decoded PowerShell Payload Output:
          </div>
          <div className="text-white break-all">{decodedOutput}</div>
        </div>
      )}
    </div>
  )}

  {/* ================= LAB ENGINE 3: BLAST CHAMBER & RE INSPECTOR (Malware) ================= */}
  {activeModule.lab_type === 'hex_inspector' && (
    <div className="bg-slate-900 border border-slate-800 rounded-lg p-4 space-y-3 font-mono text-xs">
      <div className="flex flex-wrap items-center justify-between gap-2 pb-2 border-b border-slate-800">
        <div>
          <div className="text-rose-400 font-bold">{activeModule.lab_config.sample_name}</div>
          <div className="text-[11px] text-slate-400">
            Section Entropy: {activeModule.lab_config.entropy}
          </div>
        </div>
      </div>

      {/* Inspector Sub-Navigation Tabs */}
      <div className="flex gap-1.5 bg-slate-950 p-1 rounded border border-slate-800">
        <button
          onClick={() => setMwTab('static')}
          className={`flex-1 py-1.5 rounded text-[11px] transition ${
            mwTab === 'static'
              ? 'bg-rose-500/20 text-rose-300 border border-rose-500/40'
              : 'text-slate-400 hover:text-white'
          }`}
        >
          1. Static IAT & Strings
        </button>
        <button
          onClick={() => setMwTab('sandbox')}
          className={`flex-1 py-1.5 rounded text-[11px] transition ${
            mwTab === 'sandbox'
              ? 'bg-rose-500/20 text-rose-300 border border-rose-500/40'
              : 'text-slate-400 hover:text-white'
          }`}
        >
```

```
            2. Blast Chamber Trace
          </button>
          <button
            onClick={() => setMwTab('decompiled')}
            className={`flex-1 py-1.5 rounded text-[11px] transition ${
              mwTab === 'decompiled'
                ? 'bg-rose-500/20 text-rose-300 border border-rose-500/40'
                : 'text-slate-400 hover:text-white'
            }`}
          >
            3. Decompiled RE & YARA
          </button>
        </div>

        {mwTab === 'static' && (
          <div className="grid grid-cols-1 md:grid-cols-2 gap-3">
            <div className="p-3 bg-slate-950 rounded border border-slate-800 space-y-1">
              <div className="text-slate-400 font-semibold mb-1">
                Import Address Table (IAT)
              </div>
              {(activeModule.lab_config.imports || []).map((imp, i) => (
                <div key={i} className="text-amber-300">
                  {imp}
                </div>
              ))}
            </div>
            <div className="p-3 bg-slate-950 rounded border border-slate-800 space-y-1">
              <div className="text-slate-400 font-semibold mb-1">
                Extracted Strings / IOCs
              </div>
              {(activeModule.lab_config.strings || []).map((str, i) => (
                <div key={i} className="text-emerald-300 break-all">
                  "{str}"
                </div>
              ))}
            </div>
          </div>
        )}

        {mwTab === 'sandbox' && (
          <div className="p-3 bg-slate-950 rounded border border-slate-800 space-y-1">
            <div className="text-rose-400 font-semibold mb-1">
              Isolated Blast Chamber Detonation Log
            </div>
            <pre className="text-slate-300 whitespace-pre-wrap leading-relaxed">
              {activeModule.lab_config.sandbox_trace}
            </pre>
          </div>
        )}

        {mwTab === 'decompiled' && (
          <div className="space-y-3 max-h-64 overflow-y-auto pr-1">
            <div className="p-3 bg-slate-950 rounded border border-slate-800">
              <div className="text-sky-400 font-semibold mb-1">
                Ghidra Decompiled Pseudocode & IR Playbook
              </div>
              <pre className="text-slate-300 whitespace-pre-wrap">
                {activeModule.lab_config.decompiled_c}
              </pre>
            </div>
            <div className="p-3 bg-slate-950 rounded border border-slate-800">
              <div className="text-emerald-400 font-semibold mb-1">
                Detection & Remediation YARA Rule
              </div>
              <pre className="text-slate-300 whitespace-pre-wrap">
                {activeModule.lab_config.remediation_yara}
              </pre>
            </div>
          </div>
        )}
      </div>
    )}
  </div>
```

```
        {/* Flag / IOC Submission & Verification Footer */}
        <div className="pt-4 border-t border-slate-800 space-y-3">
          <form onSubmit={handleVerify} className="flex gap-2">
            <input
              value={submission}
              onChange={(e) => setSubmission(e.target.value)}
              placeholder="Submit extracted flag, IOC, timestamp, or key..."
              className="flex-1 bg-slate-900 border border-slate-700 rounded-md px-3 py-2 text-xs md:text-sm text-
            />
            <button
              type="submit"
              className="bg-emerald-600 hover:bg-emerald-500 text-white font-medium px-4 py-2 rounded-md text-xs m
            >
              Verify Finding
            </button>
            <button
              type="button"
              onClick={() => setShowHint(!showHint)}
              className="px-3 py-2 border border-slate-700 rounded-md text-xs text-slate-300 hover:text-white hove
            >
              Hint
            </button>
          </form>

          {showHint && (
            <div className="text-xs bg-amber-500/10 border border-amber-500/30 text-amber-200 p-3 rounded-md">
              <span className="font-semibold">Lab Hint:</span> {activeModule.hint}
            </div>
          )}

          {feedback && (
            <div
              className={`text-xs p-3 rounded-md border font-mono ${
                feedback.correct
                  ? 'bg-emerald-500/10 border-emerald-500/40 text-emerald-300'
                  : 'bg-rose-500/10 border-rose-500/40 text-rose-300'
              }`}
            >
              {feedback.message}
            </div>
          )}
        </div>
      </section>
    </div>

    {/* MODAL: Research Package Ingestion Simulator */}
    {showIngestModal && (
      <div className="fixed inset-0 bg-black/80 backdrop-blur-sm flex items-center justify-center p-4 z-50">
        <div className="bg-slate-900 border border-slate-700 rounded-xl max-w-2xl w-full p-6 space-y-4">
          <div className="flex items-center justify-between">
            <div>
              <h3 className="text-base font-bold text-white">
                Phase 2 Workflow: Ingest Compiled Research Package
              </h3>
              <p className="text-xs text-slate-400">
                Paste a compiled module JSON payload (or click "Load Sample 301 Module") to hot-add it to the curr
              </p>
            </div>
            <button
              onClick={() => setShowIngestModal(false)}
              className="text-slate-400 hover:text-white text-sm"
            >
              ✕
            </button>
          </div>

          <div className="flex justify-end">
            <button
              type="button"
              onClick={loadSampleResearchPackage}
              className="text-xs bg-emerald-500/20 text-emerald-300 border border-emerald-500/40 px-3 py-1 rounded
            >
              Load Sample 301 Malware Module (PEB ROR13 API Hashing)
            </button>
```

```
                </div>

                <form onSubmit={handleIngestSubmit} className="space-y-3">
                  <textarea
                    value={ingestJson}
                    onChange={(e) => setIngestJson(e.target.value)}
                    rows={12}
                    placeholder='Paste module JSON payload here...'
                    className="w-full bg-slate-950 border border-slate-800 rounded p-3 font-mono text-xs text-emerald-30
                  />
                  <div className="flex justify-end gap-2">
                    <button
                      type="button"
                      onClick={() => setShowIngestModal(false)}
                      className="px-4 py-2 rounded text-xs text-slate-400 hover:text-white"
                    >
                      Cancel
                    </button>
                    <button
                      type="submit"
                      className="bg-emerald-600 hover:bg-emerald-500 text-white px-4 py-2 rounded text-xs font-semibold
                    >
                      Ingest into Platform
                    </button>
                  </div>
                </form>
              </div>
            </div>
          )}
        </div>
      );
    }

    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(<App />);
  </script>
</body>
</html>
```

Whenever you're ready to kick off **Phase 2**, drop in your first research package (or let me know which domain we are researching first—Penetration Testing, Threat Hunting, or Malware Engineering) and we will start building out the full module chain.

---

*User prompt: I haven't had a chance to check this yet, but I want to make sure that there is a process to record student progress through each course*

Response: Student progress tracking is built into both versions, though it handles persistence differently depending on whether you are using the single-file browser preview or the full local/Cloudflare D1 backend.

Here is how progress is currently recorded, followed by a production upgrade if you plan to support **multiple students** with persistent session resumes and CPE/completion certificates.

### 1. What Is Already Built In

- **In the Backend (`0001_schema.sql` + `src/worker/index.ts`):** The database already includes a `user_progress` table. Whenever a student submits a flag or lab finding to `POST /api/modules/:id/verify`, the Worker validates the answer server-side and automatically runs an `UPSERT` (`INSERT ... ON CONFLICT DO UPDATE`) into SQLite/D1 recording `status = 'completed'`, the total number of `attempts`, and the `completed_at` timestamp.
- **In the Single-File Preview (`secops-preview.html`):** The preview currently stores `completedIds` in React memory (which resets if you refresh the page). **Quick upgrade:** If you want the single-file preview to remember your progress across browser refreshes right now, replace two lines in `secops-preview.html`:

```
// Change this line in secops-preview.html:
const [completedIds, setCompletedIds] = useState(() => {
  const saved = localStorage.getItem('secops_completed_modules');
  return saved ? JSON.parse(saved) : ['mod-pt-101'];
});

// And add this useEffect right below it:
useEffect(() => {
  localStorage.setItem('secops_completed_modules', JSON.stringify(completedIds));
}, [completedIds]);
```

### 2. Upgrading the Backend for Multi-Student & Granular Tracking

The starter schema I wrote earlier assumes a **single local user** (`module_id` is the primary key in `user_progress`) and only records a module *after* the final flag is verified.

If you want a complete student progress system that tracks **multiple students**, records **when they start a module**, saves their **in-progress lab notes/commands**, and calculates **per-course completion percentages**, here is the upgraded schema and API logic to drop into your backend.

#### A. Upgraded Progress Schema (`migrations/0001_schema.sql` replacement for `user_progress`)

```
DROP TABLE IF EXISTS student_module_progress;
DROP TABLE IF EXISTS students;

-- 1. Student Profiles (Works locally or maps to Cloudflare Access / Zero Trust email headers)
CREATE TABLE students (
    id TEXT PRIMARY KEY,
    email TEXT UNIQUE NOT NULL,
    display_name TEXT NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    last_active_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- 2. Granular Per-Module & Per-Course Progress
CREATE TABLE student_module_progress (
    student_id TEXT NOT NULL,
    module_id TEXT NOT NULL,
    track_id TEXT NOT NULL,
    status TEXT CHECK(status IN ('not_started', 'in_progress', 'completed')) DEFAULT 'not_started',
    current_step INTEGER DEFAULT 1,         -- Tracks which theory section/sub-tab the student is on
    attempts INTEGER DEFAULT 0,             -- Counts both failed and successful flag verifications
    hints_used INTEGER DEFAULT 0,           -- Tracks if the student needed hints (for scoring)
    saved_lab_state_json TEXT DEFAULT '{}', -- Persists terminal history or scratchpad notes
    time_spent_seconds INTEGER DEFAULT 0,   -- Heartbeat timer for CPE credit verification
    started_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    last_accessed_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    completed_at DATETIME,
    PRIMARY KEY (student_id, module_id),
    FOREIGN KEY (student_id) REFERENCES students(id),
    FOREIGN KEY (module_id) REFERENCES modules(id)
);

-- Seed a default local operator profile
INSERT INTO students (id, email, display_name)
VALUES ('usr-local-01', 'operator@localhost', 'Local Operator');
```

#### B. Automatic Lifecycle Checkpoints (`src/worker/index.ts` additions)

Instead of waiting until the end of a module to write to the database, the API records progress at three distinct checkpoints:

1. **Checkpoint 1 — Module Opened (`in_progress`):** The moment a student opens a module, it logs `started_at` (if not already started) and updates `last_accessed_at`.
2. **Checkpoint 2 — State/Time Heartbeat (`PATCH /api/progress/:moduleId`):** Every 30 seconds (or when switching tabs), the client syncs `time_spent_seconds`, `hints_used`, and `saved_lab_state_json` so a student can close their laptop and resume at the exact same terminal state later.
3. **Checkpoint 3 — Flag Verification & Course Rollup (`POST /api/modules/:id/verify`):** Logs every attempt (pass or fail) and calculates overall course/track completion percentages.

```
// Helper: Resolve student identity (Uses Cloudflare Access header in prod, falls back to local operator)
const getStudentId = (c: any) => {
  return c.req.header('Cf-Access-Authenticated-User-Email') || 'usr-local-01';
};

// 1. Course-Level Progress Rollup Endpoint (Dashboard / Transcript View)
app.get('/api/progress/summary', async (c) => {
  const studentId = getStudentId(c);

  const { results } = await c.env.DB.prepare(`
    SELECT
      t.id as track_id,
      t.title as track_title,
      COUNT(m.id) as total_modules,
      SUM(CASE WHEN p.status = 'completed' THEN 1 ELSE 0 END) as completed_modules,
```

```
      SUM(CASE WHEN p.status = 'in_progress' THEN 1 ELSE 0 END) as in_progress_modules,
      COALESCE(SUM(p.time_spent_seconds), 0) as total_seconds_spent
    FROM tracks t
    LEFT JOIN modules m ON t.id = m.track_id
    LEFT JOIN student_module_progress p
      ON m.id = p.module_id AND p.student_id = ?
    GROUP BY t.id
    ORDER BY t.order_index ASC
  `).bind(studentId).all();

  return c.json({ student_id: studentId, tracks: results });
});

// 2. Auto-Start / Resume Checkpoint (Called when a module loads)
app.post('/api/progress/:moduleId/start', async (c) => {
  const studentId = getStudentId(c);
  const moduleId = c.req.param('moduleId');
  const { track_id } = await c.req.json<{ track_id: string }>();

  await c.env.DB.prepare(`
    INSERT INTO student_module_progress (student_id, module_id, track_id, status, started_at, last_accessed_at)
    VALUES (?, ?, ?, 'in_progress', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP)
    ON CONFLICT(student_id, module_id) DO UPDATE SET
      last_accessed_at = CURRENT_TIMESTAMP
  `).bind(studentId, moduleId, track_id).run();

  const saved = await c.env.DB.prepare(`
    SELECT status, current_step, attempts, hints_used, saved_lab_state_json, time_spent_seconds
    FROM student_module_progress
    WHERE student_id = ? AND module_id = ?
  `).bind(studentId, moduleId).first();

  return c.json(saved);
});

// 3. Heartbeat & Lab State Autosave (Saves terminal history, notes, and active time)
app.patch('/api/progress/:moduleId/autosave', async (c) => {
  const studentId = getStudentId(c);
  const moduleId = c.req.param('moduleId');
  const { current_step, hints_used, saved_lab_state, elapsed_seconds } = await c.req.json();

  await c.env.DB.prepare(`
    UPDATE student_module_progress
    SET current_step = ?,
        hints_used = ?,
        saved_lab_state_json = ?,
        time_spent_seconds = time_spent_seconds + ?,
        last_accessed_at = CURRENT_TIMESTAMP
    WHERE student_id = ? AND module_id = ?
  `).bind(
    current_step,
    hints_used,
    JSON.stringify(saved_lab_state || {}),
    elapsed_seconds || 0,
    studentId,
    moduleId
  ).run();

  return c.json({ saved: true });
});
```

### 3. Zero-Code Authentication When You Move to Cloudflare

Notice the `getStudentId` helper above checks for the `Cf-Access-Authenticated-User-Email` header.

- **Locally:** It automatically logs everything under `usr-local-01` without nagging you for a login screen while you build and test courses.
- **On Cloudflare:** When you eventually deploy this to Cloudflare, you can toggle on **Cloudflare Zero Trust Access** (which is free for up to 50 users and supports Google, GitHub, or one-time PIN login) in front of your domain. Cloudflare will automatically inject the authenticated student's email header into every Worker request—giving you multi-student progress tracking without having to maintain password hashing or session cookies in your own code.

Would you like me to update the single-file HTML preview to include a **Student Progress & CPE Transcript Drawer** (with `localStorage` persistence so your progress saves between refreshes), or keep that focused on the backend files?

---